

Lecture 02: Analysis of Algorithms

INF1008 Data Structures and Algorithms

Written By: Timothy Chia

Contents

1	Introduction to Algorithm Analysis	2
1.1	Order of Growth	2
2	Mathematics for Algorithms	3
2.1	Polynomials and dominance of the highest-degree term	3
2.2	Combinations and loop counting (1-Sum and 2-Sum)	3
2.3	Logarithms and repeated halving	4
2.4	Summation of series and algorithm costs	6
3	Empirical vs Theoretical Analysis	6
4	Computational Models	7
4.1	RAM model	7
4.2	Operation counting with 1-Sum and 2-Sum	7
5	Asymptotic Analysis	9
5.1	Motivation: growth for large inputs and worst-case focus	9
5.2	Big-O: formal definition and intuition	9
5.3	Big-Omega and Big-Theta	10
6	Combining and Simplifying Growth	10
6.1	Sequential components	10
6.2	Nested components	10
6.3	Dominant term analysis	11
7	Common Orders of Growth	11
8	Types of Algorithm Analysis	11
9	Theory of Algorithms	12

1 Introduction to Algorithm Analysis

An **algorithm** is a finite, unambiguous sequence of steps that transforms an *input* into an *output* and is guaranteed to terminate. In computing, correctness is only the first requirement. We also care about *efficiency*: whether the algorithm remains usable when the input becomes large.

Algorithm analysis answers questions such as:

- If the input size doubles, does the running time roughly double, quadruple, or become completely infeasible?
- Is the algorithm limited by *time* (too slow) or *space* (uses too much memory)?

Time complexity describes how the running time grows with the input size n . **Space complexity** describes how memory usage grows with n . The goal is not to predict the exact number of milliseconds on one machine, but to understand *scalability*. Small constant differences matter for tiny inputs; growth rates dominate for large inputs.

A useful mindset is: *efficiency is about the shape of growth*. This is why we develop mathematical tools to count how often key operations run as n increases.

1.1 Order of Growth

Order of growth compares how resources scale as the *volume* of input grows. A simple way to build intuition is to contrast a *bundled* process that has a fixed overhead with a *per-item* process whose cost increases with the number of items. For instance, suppose a bundled method takes a fixed 24 hours regardless of how many items are requested, while a per-item method takes about 2 hours per item, giving $T(n) = 2n$ hours. The exact numbers are not the point; the growth pattern is. When n doubles, the bundled time stays the same, while the per-item time doubles.

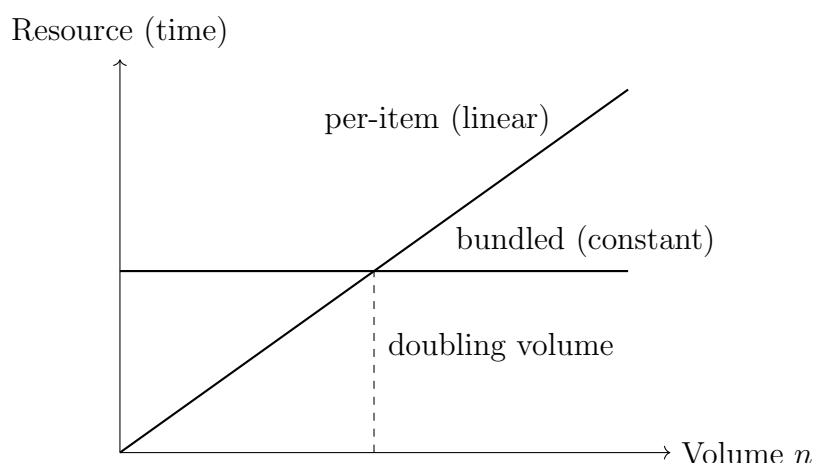


Figure 1: Constant vs linear growth: the bundled method is $O(1)$ in volume; the per-item method is $O(n)$.

2 Mathematics for Algorithms

This section collects the core mathematical ideas used to analyse loops and recursion. Each concept will reappear later when we count operations and simplify complexity expressions.

2.1 Polynomials and dominance of the highest-degree term

A polynomial has the form

$$p(n) = a_d n^d + a_{d-1} n^{d-1} + \cdots + a_1 n + a_0,$$

where d is the degree and $a_d \neq 0$. For large n , the term $a_d n^d$ dominates the growth because n^d eventually overwhelms every lower power.

To see this, divide by the highest power:

$$\frac{p(n)}{n^d} = a_d + \frac{a_{d-1}}{n} + \cdots + \frac{a_0}{n^d} \xrightarrow{n \rightarrow \infty} a_d.$$

So, for large n , $p(n)$ behaves like $a_d n^d$ up to constant factors.

Table 1: Dominant-term intuition for common polynomials.

Polynomial $p(n)$	Dominant behaviour as n grows
	$3n + 7$
behaves like n (linear)	behaves like n^2 (quadratic)
$2n^2 + 3n + 3$	$n^3 + 100n^2 + 7$
behaves like n^3 (cubic)	behaves like n^4 (quartic)
$5n^4 + n$	

2.2 Combinations and loop counting (1-Sum and 2-Sum)

A **combination** counts ways to choose k items from n items without caring about order:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

This connects naturally to algorithms that examine all pairs, triples, or subsets of elements.

1-Sum (single loop). If a loop performs constant work once per iteration from 1 to n , the total work is

$$\sum_{i=1}^n 1 = n.$$

The key idea: one loop with n iterations typically gives linear growth.

2-Sum (triangular nested loop). A common nested-loop pattern is:

```
for i in range(1, n+1):
    for j in range(i+1, n+1):
        # constant work
        pass
```

For each fixed i , the inner loop runs $n - i$ times. Total inner executions:

$$\begin{aligned}\sum_{i=1}^n (n - i) &= (n - 1) + (n - 2) + \cdots + 1 \\ &= \frac{n(n - 1)}{2} \\ &= \binom{n}{2}.\end{aligned}$$

This triangular count is the reason “all-pairs” style algorithms are typically quadratic.

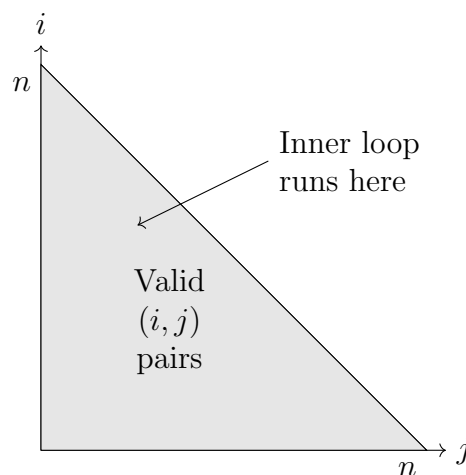


Figure 2: The 2-Sum nested loop counts a triangular region of index pairs, giving $\frac{n(n-1)}{2}$ inner executions.

2.3 Logarithms and repeated halving

A logarithm answers: “what exponent produces n ?”

$$\log_b(n) = x \iff b^x = n, \quad b > 0, \quad b \neq 1, \quad n > 0.$$

Logarithms appear whenever an algorithm repeatedly *halves* (or, more generally, divides by a constant factor) the problem size.

Repeated halving. Suppose you start with n and keep dividing by 2 until reaching 1:

$$n, \frac{n}{2}, \frac{n}{4}, \dots, 1.$$

After k steps you have $n/2^k$. Set $n/2^k = 1$ to get $2^k = n$, so $k = \log_2 n$. This is why binary search runs in $O(\log n)$: each step removes about half the remaining candidates.

Table 2: Useful logarithm laws for simplifying runtime expressions.

Law	Meaning
$\log_b(xy) = \log_b x + \log_b y$	products become sums
$\log_b(x/y) = \log_b x - \log_b y$	quotients become differences
$\log_b(x^k) = k \log_b x$	exponents become multipliers
$\log_a n = \frac{\log_b n}{\log_b a}$	change of base (differs by a constant factor)

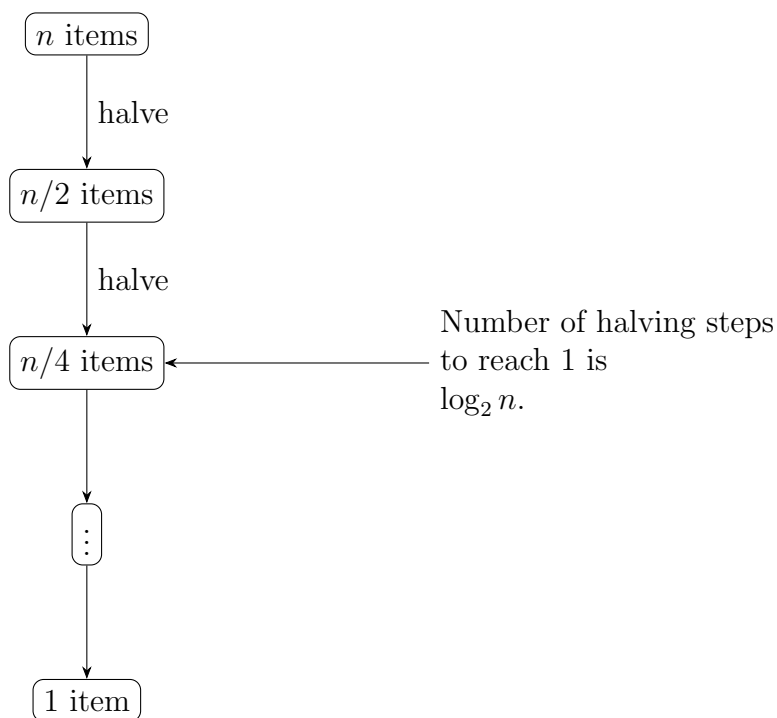


Figure 3: Repeated halving leads to logarithmic time.

2.4 Summation of series and algorithm costs

Many runtimes are expressed as summations, especially when counting loop iterations.

Arithmetic series.

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}.$$

This is the mathematical reason triangular nested loops often yield quadratic growth.

Geometric series. For ratio $r \neq 1$,

$$\sum_{i=0}^k r^i = \frac{r^{k+1} - 1}{r - 1}.$$

When $r = 2$, $1 + 2 + 4 + \dots + 2^k = 2^{k+1} - 1$. Geometric sums frequently appear in divide-and-conquer recurrences and in processes that repeatedly double/halve.

Table 3: Series formulas that commonly appear in loop and recursion analysis.

Series	Closed form
$\sum_{i=1}^n 1$	n
$\sum_{i=1}^n i$	$\frac{n(n+1)}{2}$
$\sum_{i=0}^k r^i \ (r \neq 1)$	$\frac{r^{k+1} - 1}{r - 1}$
$\sum_{i=0}^k 2^i$	$2^{k+1} - 1$

3 Empirical vs Theoretical Analysis

There are two broad ways to evaluate an algorithm.

Empirical analysis measures time and memory by running real code on a real system. This is excellent for engineering decisions (choice of language, data layout, constant-factor improvements), but it is difficult to generalise. Measurements depend on CPU speed, caching, compiler optimisations, the operating system, background processes, and the input distribution.

Theoretical analysis uses a simplified computational model and counts abstract operations as a function of input size n . The advantage is portability: the result is a machine-independent prediction of scalability. Theoretical models are especially valuable when inputs can grow far beyond what is comfortable to benchmark.

Table 4: Empirical vs theoretical analysis: what each approach tells you.

Empirical (measurements)	Theoretical (models)
Depends on hardware/software settings	Independent of specific machines
Captures constant factors and overhead	Focuses on growth rates
Requires implementation and testing	Enables early comparison before coding
Sensitive to input distribution used	Usually analyses worst-case or general n
Best for optimisation <i>in practice</i>	Best for predicting <i>scalability</i>

4 Computational Models

To analyse algorithms theoretically, we need a model of what counts as “work”.

4.1 RAM model

The **RAM model** assumes:

1. Each elementary operation takes constant time.
2. Accessing any memory location takes constant time.
3. The cost of an algorithm is approximated by counting elementary operations.

Under this model, operations such as assignment, arithmetic, comparisons, and array indexing are treated as constant time.

Table 5: Examples of elementary operations (each treated as $O(1)$ in the RAM model).

Category	Examples
Arithmetic	$+$, $-$, $\times$, $\div$ on fixed-size numbers
Assignment	$x \leftarrow y$, $x \leftarrow c$
Comparison	$x < y$, $x == y$
Logic/branching	if , else , while condition checks
Memory access	reading/writing $A[i]$

4.2 Operation counting with 1-Sum and 2-Sum

Operation counting converts code into a cost function $T(n)$. A consistent method is:

1. Identify the input size n .
2. Choose which statements count as unit-cost operations.
3. Count how often each statement executes.
4. Sum the contributions to obtain $T(n)$, then simplify.

Example: 1-Sum. Consider:

```
def one_sum(A):
    s = 0
    for i in range(len(A)):
        s += A[i]
    return s
```

If we count a constant number of operations per loop iteration, the total is of the form

$$T(n) = an + b,$$

for some constants $a, b > 0$. This is linear growth: doubling n roughly doubles $T(n)$.

Example: counting an iterative factorial. Consider:

```
def factorial(n):
    res, val = 1, 1
    while val <= n:
        res = res * val
        val = val + 1
    return res
```

If we treat assignment, comparison, addition, and multiplication as unit-cost steps, then each loop iteration performs a constant number of such steps. The loop runs n times (plus one final condition check), so the total cost is linear: $T(n) = an + b$ for constants $a, b > 0$, hence $T(n) = O(n)$.

Table 6: Illustrative step count for iterative factorial under a simple unit-cost model.

n	steps $T(n)$ (example)	growth
1	$5(1) + 4 = 9$	
2	$5(2) + 4 = 14$	
3	$5(3) + 4 = 19$	$\propto n$
4	$5(4) + 4 = 24$	

Example: 2-Sum. Consider:

```
def two_sum(A):
    s = 0
    n = len(A)
    for i in range(n):
        for j in range(i+1, n):
            s += A[i] + A[j]
    return s
```

The inner loop runs $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$ times. If each inner iteration performs constant work, then the dominant term of $T(n)$ is proportional to n^2 .

Table 7: Counting inner-loop executions for the 2-Sum pattern.

Outer index i	Inner iterations $n - i - 1$	Contribution
0	$n - 1$	$n - 1$
1	$n - 2$	$n - 2$
$\vdots$	$\vdots$	$\vdots$
$n - 2$	1	1
$n - 1$	0	0
Total	$\sum_{i=0}^{n-1} (n - i - 1) = \frac{n(n-1)}{2}$	

5 Asymptotic Analysis

Exact counts (like $3n + 3$ or $2n^2 + 3n + 3$) are useful, but algorithm comparison is usually driven by how $T(n)$ grows for large n . **Asymptotic analysis** describes growth while ignoring constant factors and lower-order terms.

5.1 Motivation: growth for large inputs and worst-case focus

As n grows, the dominant term decides feasibility. For example, n^2 grows faster than $n \log n$, and both grow far slower than 2^n . This is why we focus on large-input behaviour rather than small-input timing.

Worst-case analysis is commonly used because it provides a performance guarantee: for inputs of size n , the algorithm will not exceed the derived upper bound.

5.2 Big-O: formal definition and intuition

Big-O provides an asymptotic upper bound.

$$f(n) = O(g(n)) \iff \exists k > 0, \exists n_0 \text{ s.t. } \forall n \geq n_0, f(n) \leq k g(n).$$

Intuitively, once n is large enough, $g(n)$ (up to a constant multiple) sits above $f(n)$.

Worked proof (1-Sum). Let $f(n) = 3n + 3$. We show $f(n) = O(n)$. For $n \geq 1$, $3 \leq 3n$, so

$$3n + 3 \leq 3n + 3n = 6n.$$

Choose $k = 6$ and $n_0 = 1$. Then $f(n) \leq kn$ for all $n \geq n_0$, hence $3n + 3 = O(n)$.

Worked proof (2-Sum). Let $f(n) = 2n^2 + 3n + 3$. We show $f(n) = O(n^2)$. For $n \geq 1$, $3n \leq 3n^2$ and $3 \leq 3n^2$, so

$$2n^2 + 3n + 3 \leq 2n^2 + 3n^2 + 3n^2 = 8n^2.$$

Choose $k = 8$ and $n_0 = 1$. Then $f(n) \leq kn^2$ for all $n \geq n_0$, hence $f(n) = O(n^2)$.

5.3 Big-Omega and Big-Theta

Big-Omega is an asymptotic lower bound:

$$f(n) = \Omega(g(n)) \iff \exists k > 0, \exists n_0 \text{ s.t. } \forall n \geq n_0, f(n) \geq k g(n).$$

Big-Theta is a tight bound (both upper and lower):

$$f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)).$$

Interpretation. If $T(n) = O(g(n))$, the algorithm grows no faster than $g(n)$ up to constants. If $T(n) = \Omega(g(n))$, it grows at least as fast as $g(n)$. If $T(n) = \Theta(g(n))$, then $g(n)$ captures the true growth rate (within constant factors).

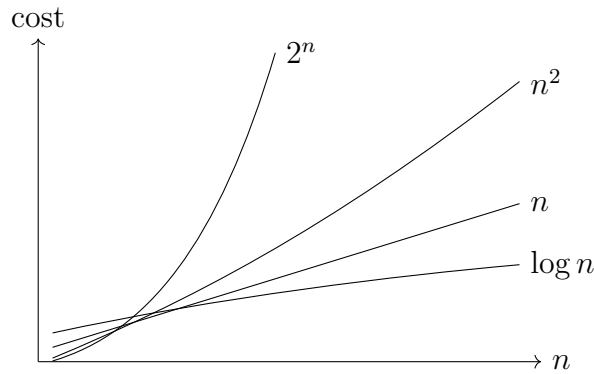


Figure 4: Qualitative growth-rate comparison: logarithmic $\ll$ linear $\ll$ quadratic $\ll$ exponential.

6 Combining and Simplifying Growth

Real algorithms combine multiple parts. To compute a clean asymptotic bound, we use a small set of rules.

6.1 Sequential components

If an algorithm runs part A then part B, the total time is

$$T(n) = T_A(n) + T_B(n).$$

As n grows, the dominant term determines the final class. For example, $n^2 + 10n$ behaves like n^2 .

6.2 Nested components

If a loop of n iterations contains an inner process of cost $T_{\text{inner}}(n)$, the total often multiplies:

$$T(n) = n \cdot T_{\text{inner}}(n).$$

For example, an outer loop n times and an inner loop n times gives $n \cdot n = n^2$.

6.3 Dominant term analysis

Two simplifications are standard:

1. **Ignore constant factors:** $c \cdot g(n)$ is the same growth class as $g(n)$ for any constant $c > 0$.
2. **Ignore lower-order terms:** when adding terms, keep only the fastest-growing term.

So $2n^2 + 3n + 3$ simplifies to $\Theta(n^2)$. This is not claiming constants are meaningless; it is claiming they do not change the growth family.

7 Common Orders of Growth

The most common growth classes appear across sorting, searching, graph algorithms, and dynamic programming.

Table 8: Common orders of growth with intuition and typical examples.

Class	Name	Typical pattern	Scaling intuition
$O(1)$	constant	array index access, hash lookup (idealised)	cost unchanged as n grows
$O(\log n)$	logarithmic	binary search, balanced BST operations	each step halves remaining work
$O(n)$	linear	scanning a list, finding max	one pass over the data
$O(n \log n)$	linearithmic	merge sort, heapsort	$\log n$ levels $\times$ n work per level
$O(n^2)$	quadratic	all pairs, simple nested loops	work for every pair of items
$O(n^3)$	cubic	triple nested loops, naive matrix multiplication	work for every triple
$O(2^n)$	exponential	brute force over all subsets	doubling n squares the search space size

A helpful practical perspective is: $O(n)$ and $O(n \log n)$ often scale well; $O(n^2)$ becomes heavy when n is large; exponential growth becomes infeasible extremely quickly.

8 Types of Algorithm Analysis

An algorithm can behave differently depending on the input. Therefore, analysis often considers multiple cases:

Best-case is the minimum running time over all inputs of size n . **Worst-case** is the maximum. **Average-case** is the expected running time under an assumed probability distribution over inputs.

Worst-case analysis is commonly used because it provides a reliable guarantee, especially when inputs are unpredictable or adversarial.

Table 9: Best-, average-, and worst-case illustrated with a simple search task.

Case	Example: linear search for a target in an array of length n
Best-case	target is at the first position $\Rightarrow$ 1 comparison $\Rightarrow O(1)$
Average-case	target is equally likely to be in any position $\Rightarrow$ about $\frac{n}{2}$ comparisons $\Rightarrow O(n)$
Worst-case	target is absent or at the last position $\Rightarrow n$ comparisons $\Rightarrow O(n)$

Different algorithms can have the same worst-case class but different practical behaviour; likewise, some algorithms have excellent average-case performance yet poor worst-case guarantees. In a first analysis, worst-case bounds are the standard baseline.

9 Theory of Algorithms

Algorithm theory connects *what is achievable* with *what is optimal*. Two ideas are central:

Upper bounds. An algorithm has an upper bound $O(g(n))$ if we can prove it never grows faster than $g(n)$ (up to constants) for sufficiently large n .

Lower bounds. A problem has a lower bound $\Omega(h(n))$ under a given computational model if no algorithm can solve the problem asymptotically faster than $h(n)$ in the worst case.

Optimal algorithms. An algorithm is **asymptotically optimal** (for a model and problem) if its upper bound matches a proven lower bound up to constant factors, i.e. it achieves $\Theta(\cdot)$ equal to the lower bound. This matters because it tells you when further improvement requires changing assumptions (e.g. randomness, extra memory, approximation) rather than simply “coding better”.

From theory to practice. In real systems, you still care about constants, memory limits, and typical inputs. However, asymptotic bounds remain the first filter for selecting algorithms that will not collapse when data grows. Once you have the right growth class, implementation details and empirical testing become the next step for performance tuning.